

LAPORAN PRAKTIKUM PEKAN 9
STRUKTUR DATA

Disusun Oleh:

Syasya Halwa Gazwani

(2511531018)

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Muhammad Galid Alvero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG
2026

KATA PENGANTAR

Assalamu'alaikum warahmatullahi wabarakatuh,

Segala puji bagi Allah SWT yang telah memberikan kesempatan serta kemudahan kepada penulis untuk dapat menyelesaikan Laporan Praktikum pada mata kuliah Struktur Data, sehingga Laporan Praktikum ini dapat dikumpulkan dengan tepat waktu. Atas rahmat dan karunianya Laporan Praktikum dapat terselesaikan dengan baik. Sholawat serta salam juga penulis sampaikan kepada baginda tercinta yaitu Nabi Muhammad SAW. yang kita nantikan syafa'atnya di akhirat nanti. Laporan Praktikum ini bertujuan untuk menambah wawasan para pembaca untuk lebih memperdalam ilmu yang ada pada makalah ini.

Dalam penyusunan Laporan Praktikum ini, penulis mengalami banyak kesulitan dan penulis menyadari bahwa Laporan Praktikum ini masih jauh dari kesempurnaan. Untuk itu, penulis sangat mengharapkan kritik dan saran yang membangun demi kesempurnaan pada Laporan Praktikum ini. Penulis mengucapkan terima kasih kepada bapak Dr. Wahyudi, S.T, M.T. selaku dosen mata kuliah Struktur Data yang telah memberikan tugas ini sehingga dapat menambah pengetahuan dan wawasan sesuai dengan mata kuliah yang penulis tekuni. Penulis juga mengucapkan terima kasih kepada semua pihak yang telah membagi sebagian pengetahuannya sehingga penulis dapat menyelesaikan Laporan Praktikum ini.

Padang, Juni 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN	3
DASAR TEORI	3
2.1 Node_2511531018.....	3
2.1.1 Kode Program	3
2.1.2 Langkah Kerja	5
2.1.3 Output	8
2.2 BTree_2511531018.....	8
2.2.1. Kode Program	8
2.2.2. Langkah Kerja	9
2.3 BtreeDriver_2511531018	12
2.3.1 Kode Program	12
2.3.2 Langkah Kerja	13
2.4.3 Output	15
2.4 GraphTraversal_2511531018	16
2.4.1 Kode Program	16
2.4.2 Langkah Kerja	17
2.4.3 Output	20
2.5 TUGAS.....	21

2.6.1	PetaMall_2511531018	21
2.6.2	Output	27
2.6.3	Penjelasan	27
BAB III PENUTUP		31
DAFTAR PUSTAKA.....		32

BAB I PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu konsep penting dalam pemrograman yang digunakan untuk mengelola dan menyimpan data secara terorganisir agar dapat diakses dan diproses dengan lebih efisien. Beberapa struktur data yang sering digunakan adalah tree dan graph. Binary Tree digunakan untuk menyimpan data dalam bentuk hierarki yang terdiri dari root, parent, dan child node, sedangkan Graph digunakan untuk merepresentasikan hubungan antar simpul yang saling terhubung.

Dalam implementasinya, Binary Tree memiliki beberapa metode penelusuran seperti Preorder, Inorder, dan Postorder untuk mengunjungi setiap simpul pada pohon. Sementara itu, pada Graph terdapat algoritma penelusuran Depth First Search (DFS) dan Breadth First Search (BFS) yang digunakan untuk menjelajahi seluruh simpul dalam graf. Oleh karena itu, praktikum ini dilakukan untuk memahami cara pembuatan Binary Tree dan Graph serta mengimplementasikan berbagai metode traversal yang terdapat pada kedua struktur data tersebut menggunakan bahasa pemrograman Java.

1.2 Tujuan

- 1.2.1 Memahami konsep dasar struktur data graf dan representasinya menggunakan adjacency list.
- 1.2.2 Mengimplementasikan algoritma Depth First Search (DFS) dan Breadth First Search(BFS) pada graf.
- 1.2.3 Menganalisis perbedaan cara kerja DFS dan BFS dalam melakukan penelusuran simpul.

1.3 Manfaat Praktikum

- 1.3.1 Menambah pemahaman tentang struktur data graf dan penggunaannya.
- 1.3.2 Membantu memahami perbedaan karakteristik DFS dan BFS.

- 1.2.4 Meningkatkan kemampuan berpikir logis dalam menyelesaikan permasalahan yang melibatkan hubungan antar simpul.

BAB II PEMBAHASAN

DASAR TEORI

Struktur data merupakan cara untuk mengorganisasi, menyimpan, dan mengelola data di dalam komputer agar dapat diakses dan diproses secara efisien. Pada praktikum ini digunakan dua struktur data, yaitu Binary Tree dan Graph. Binary Tree adalah struktur data yang terdiri dari kumpulan node, di mana setiap node memiliki paling banyak dua anak, yaitu anak kiri dan anak kanan. Pada Binary Tree terdapat beberapa metode traversal yang digunakan untuk mengunjungi seluruh node, yaitu Preorder, Inorder, dan Postorder. Proses traversal umumnya dilakukan menggunakan teknik rekursi karena setiap subtree memiliki struktur yang sama dengan tree secara keseluruhan. Selain itu, Binary Tree juga dapat digunakan untuk melakukan pencarian data dan menghitung jumlah node yang terdapat dalam pohon.

Graph merupakan struktur data yang terdiri dari sekumpulan simpul (vertex) dan hubungan antar simpul yang disebut sisi (edge). Pada praktikum ini, graph direpresentasikan menggunakan adjacency list, yaitu setiap simpul menyimpan daftar simpul lain yang terhubung langsung dengannya. Untuk melakukan penelusuran pada graph digunakan algoritma Depth First Search (DFS) dan Breadth First Search (BFS). DFS bekerja dengan mengunjungi simpul sedalam mungkin terlebih dahulu sebelum kembali ke simpul sebelumnya, sedangkan BFS melakukan penelusuran berdasarkan tingkat atau level kedekatan dari simpul awal menggunakan struktur data queue. Kedua algoritma tersebut banyak digunakan dalam berbagai aplikasi seperti pencarian jalur, analisis jaringan, dan pemetaan hubungan antar data.

2.1 Node_2511531018

2.1.1 Kode Program

```
package pekan9_2511531018;  
  
public class Node_2511531018 {
```

```

int data_1018;
Node_2511531018 left_1018;
Node_2511531018 right_1018;
public Node_2511531018(int data_1018) {
    this.data_1018 = data_1018;
    left_1018 = null;
    right_1018 = null;
}
public void setLeft_1018(Node_2511531018 node) {
    if (left_1018 == null)
        left_1018 = node;
}
public void setRight_1018(Node_2511531018 node) {
    if (right_1018 == null)
        right_1018 = node;
}
public Node_2511531018 getLeft_1018() {
    return left_1018;
}
public Node_2511531018 getRight_1018() {
    return right_1018;
}

public int getData_1018() {
    return data_1018;
}
public void setData_1018(int data_1018) {
    this.data_1018 = data_1018;
}
void printPreorder_1018(Node_2511531018 node) {
    if (node == null)
        return;
    System.out.print(node.data_1018 + " ");
    printPreorder_1018(node.left_1018);
    printPreorder_1018(node.right_1018);
}
void printPostorder_1018(Node_2511531018 node) {
    if (node == null)
        return;
    printPostorder_1018(node.left_1018);
    printPostorder_1018(node.right_1018);
    System.out.print(node.data_1018 + " ");
}
void printInorder_1018(Node_2511531018 node) {
    if (node == null)
        return;
    printInorder_1018(node.left_1018);
    System.out.print(node.data_1018 + " ");
    printInorder_1018(node.right_1018);
}
public String print_1018() {
    return this.print_1018("", true, "");
}
public String print_1018(String prefix_1018, boolean
isTail_1018, String sb_1018) {
    if (right_1018 != null) {

```

```

        right_1018.print_1018(prefix_1018 +
(isTail_1018 ? "|" : " "), false, sb_1018);
    }
    System.out.println( prefix_1018+(isTail_1018 ? "\\--
" : "/-- ") +data_1018);
    if (left_1018 != null) {
        left_1018.print_1018(prefix_1018+(isTail_1018
? " " : "|"), true, sb_1018);
    }
    return sb_1018;
}
}

```

Gambar 2.1.1 Kode program Node_2511531018.

2.1.2 Langkah Kerja

2.1.2.1 Mendeklarasikan variabel yang dimiliki tiap node. Class node digunakan untuk membuat node pada Binary Tree. Nilai yang diberikan akan disimpan ke data_1018, sedangkan anak kiri dan kanan diisi null karena pada awalnya node belum memiliki anak.

```

public class Node_2511531018 {
    int data_1018;
    Node_2511531018 left_1018;
    Node_2511531018 right_1018;
    public Node_2511531018(int data_1018) {
        this.data_1018 = data_1018;
        left_1018 = null;
        right_1018 = null;
    }
}

```

Gambar 2.1.2.1 Method Node_2511531018.

2.1.2.2 Method digunakan untuk menambahkan anak kiri. Node baru hanya akan dimasukkan jika posisi kiri masih kosong.

```

public void setLeft_1018(Node_2511531018 node) {
    if (left_1018 == null)
        left_1018 = node;
}

```

Gambar 2.1.2.2 Method setLeft_1018.

2.1.2.3 Method ini digunakan untuk menambahkan anak kanan. Node baru hanya akan dimasukkan jika posisi kanan masih kosong,

```

public void setRight_1018(Node_2511531018 node) {
    if (right_1018 == null)
        right_1018 = node;
}

```

Gambar 2.1.2.3 Method setRight_1018.

- 2.1.2.4 Digunakan untuk mengambil atau mengembalikan node anak kiri.

```
public Node_2511531018 getLeft_1018() {
    return left_1018;
}
```

Gambar 2.1.2.4 Method getLeft_1018.

- 2.1.2.5 Digunakan untuk mengambil node anak kanan.

```
public Node_2511531018 getRight_1018() {
    return right_1018;
}
```

Gambar 2.1.2.5 Method getRight_1018.

- 2.1.2.6 Digunakan untuk mengambil nilai data yang ada pada node dan untuk mengubah nilai data pada node.

```
public int getData_1018() {
    return data_1018;
}
public void setData_1018(int data_1018) {
    this.data_1018 = data_1018;
}
```

Gambar 2.1.2.6 Method getData_1018 dan Method setData_1018.

- 2.1.2.7 Traversal preorder mengunjungi node dengan urutan yaitu **Root => Left => Right**. Pertama data node dicetak, lalu berpindah ke anak kiri, kemudian ke anak kanan.

```
void printPreorder_1018(Node_2511531018 node) {
    if (node == null)
        return;
    System.out.print(node.data_1018 + " ");
    printPreorder_1018(node.left_1018);
    printPreorder_1018(node.right_1018);
}
```

Gambar 2.1.2.7 Traversal Preorder.

- 2.1.2.8 Traversal postorder mengunjungi node dengan urutan yaitu **Left => Right => Root**. Anak kiri dan anak kanan dikunjungi terlebih dahulu, kemudian data node dicetak.

```
void printPostorder_1018(Node_2511531018 node) {
    if (node == null)
        return;
    printPostorder_1018(node.left_1018);
    printPostorder_1018(node.right_1018);
    System.out.print(node.data_1018 + " ");
}
```

Gambar 2.1.2.8 Traversal Postorder.

- 2.1.2.9 Traversal inorder mengunjungi node dengan urutan yaitu **Left** => **Root** => **Right**. Anak kiri dikunjungi terlebih dahulu, kemudian data node dicetak, lalu berpindah ke anak kanan.

```
void printInorder_1018(Node_2511531018 node) {
    if (node == null)
        return;
    printInorder_1018(node.left_1018);
    System.out.print(node.data_1018 + " ");
    printInorder_1018(node.right_1018);
}
```

Gambar 2.1.2.9 Traversal Inorder.

- 2.1.2.10 Berfungsi untuk memanggil method print_1018 yang lebih lengkap dengan parameter awal yang sudah ditentukan.

```
public String print_1018() {
    return this.print_1018("", true, "");
}
```

Gambar 2.1.2.10 Method print.

- 2.1.2.11 Method ini digunakan untuk menampilkan struktur tree secara visual seperti cabang pohon. Jika node memiliki anak kanan, method akan dipanggil secara rekursif untuk menampilkan subtree kanan terlebih dahulu.

```
public String print_1018(String prefix_1018, boolean
isTail_1018, String sb_1018) {
    if (right_1018 != null) {
        right_1018.print_1018(prefix_1018 +
(isTail_1018 ? "| " : " "), false, sb_1018);
    }
```

Gambar 2.1.2.11 Method print_1018(String prefix_1018, boolean isTail_1018, String sb_1018).

- 2.1.2.12 Mencetak data node beserta simbol cabang agar bentuk tree terlihat jelas. Setelah itu, subtree kiri dicetak.

```
System.out.println( prefix_1018+(isTail_1018 ? "\\-- "
: "/-- ") + data_1018);
    if (left_1018 != null) {
        left_1018.print_1018(prefix_1018+(isTail_1018 ? "
" : "| "), true, sb_1018);
    }
    return sb_1018;
}
```

Gambar 2.1.2.12 Mencetak sisi kanan dan sisi kiri.

2.1.3 Output

```
Graf Awal adalah:
Graf Awal (Adjacency List):
A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B

Penelusuran DFS:
A B D E C
Penelusuran BFS:
A B C D E
```

Gambar 2.1.3 Output Node_2511531018.

2.2 BTree_2511531018

2.2.1. Kode Program

```
package pekan9_2511531018;

public class BTree_2511531018 {
    private Node_2511531018 root_1018;
    private Node_2511531018 currentNode_1018;
    public BTree_2511531018() {
        root_1018 = null;
    }
    public boolean search_1018(int data_1018) {
        return search_1018(root_1018, data_1018);
    }
    private boolean search_1018(Node_2511531018 node, int data_1018) {
        if (node.getData_1018() == data_1018)
            return true;
        if (node.getLeft_1018() != null)
            if (search_1018(node.getLeft_1018(), data_1018))
                return true;
        if (node.getRight_1018() != null)
            if (search_1018(node.getRight_1018(), data_1018))
                return true;
        return false;
    }
    public void printInorder_1018() {
        root_1018.printInorder_1018(root_1018);
    }
    public void printPreorder_1018() {
        root_1018.printPreorder_1018(root_1018);
    }
    public void printPostOrder_1018() {
        root_1018.printPostorder_1018(root_1018);
    }
}
```

```

public Node_2511531018 getRoot_1018() {
    return root_1018;
}

public boolean isEmpty_1018() {
    return root_1018 == null;
}

public int countNodes_1018() {
    return countNodes_1018(root_1018);
}

private int countNodes_1018(Node_2511531018 node) {
    int count_1018 = 1;
    if (node == null) {
        return 0;
    } else {
        count_1018 +=
countNodes_1018(node.getLeft_1018());
        count_1018 +=
countNodes_1018(node.getRight_1018());
        return count_1018;
    }
}

public void print_1018() {
    root_1018.print_1018();
}

public Node_2511531018 getCurrent_1018() {
    return currentNode_1018;
}

public void setCurrent_1018(Node_2511531018 node) {
    this.currentNode_1018 = node;
}

public void setRoot_1018(Node_2511531018 root_1018) {
    this.root_1018 = root_1018;
}
}

```

Gambar 2.2.1 Kode program BTree_2511531018.

2.2.2. Langkah Kerja

2.2.2.1 Deklarasi atribut untuk menyimpan node akar (root) dari tree dan menyimpan node yang sedang aktif atau sedang diproses. Konstruktor dijalankan saat objek Btree_2511531018 dibuat. Nilai root diisi null karena tree masih kosong dan belum memiliki node.

```

public class BTree_2511531018 {
    private Node_2511531018 root_1018;
    private Node_2511531018 currentNode_1018;
    public BTree_2511531018() {

```

```
root_1018 = null;
```

Gambar 2.2.2.1 Deklarasi dan Konstruktor.

- 2.2.2.2 Method digunakan untuk mencari data dalam tree. Pencarian dimulai dari root lalu memanggil method rekursif dibawahnya.

```
public boolean search_1018(int data_1018) {
    return search_1018(root_1018, data_1018);
}
```

Gambar 2.2.2.2 Method search.

- 2.2.2.3 Jika data pada node saat ini sama dengan data yang dicari, maka pencarian berhasil dan mengembalikan nilai true.

```
private boolean search_1018(Node_2511531018 node, int data_1018) {
    if (node.getData_1018() == data_1018)
        return true;
}
```

Gambar 2.2.2.3 Proses pencarian.

- 2.2.2.4 Jika ada anak kiri, program akan mencari secara rekursif ke subtree kiri.

```
if (node.getLeft_1018() != null)
    if (search_1018(node.getLeft_1018(), data_1018))
        return true;
```

Gambar 2.2.2.4 Mencari secara rekursif.

- 2.2.2.5 Jika data belum ditemukan di kiri, program akan melanjutkan pencarian ke subtree kanan. Jika seluruh node sudah diperiksa dan data tidak ditemukan, maka method mengembalikan nilai false.

```
if (node.getRight_1018() != null)
    if (search_1018(node.getRight_1018(), data_1018))
        return true;
return false;
```

Gambar 2.2.2.5 Mencari secara rekursif.

- 2.2.2.6 Menampilkan traversal Inorder, Preorder, dan Postorder sesuai dengan urutan masing-masing traversal.

```
public void printInorder_1018() {
    root_1018.printInorder_1018(root_1018);
}
public void printPreorder_1018() {
    root_1018.printPreorder_1018(root_1018);
}
```

```

}
public void printPostOrder_1018() {
    root_1018.printPostorder_1018(root_1018);
}

```

Gambar 2.2.2.6 Traversal Tree.

- 2.2.2.7 Method `getRoot` digunakan untuk mengambil node root dari tree, Method `isEmpty` digunakan untuk mengecek apakah tree kosong atau tidak. Jika root bernilai null, maka tree masih kosong sehingga method mengembalikan true. Method `countNodes` digunakan untuk menghitung jumlah seluruh node dalam tree.

```

public Node_2511531018 getRoot_1018() {
    return root_1018;
}
public boolean isEmpty_1018() {
    return root_1018 == null;
}

public int countNodes_1018() {
    return countNodes_1018(root_1018);
}

```

Gambar 2.2.2.7 Method `getRoot`.

- 2.2.2.8 Method rekursif untuk menghitung node. Jika node kosong, jumlah node adalah 0. Setiap node yang dikunjungi dihitung sebagai 1, kemudian program menghitung seluruh node di subtree kiri dan subtree kanan.

```

private int countNodes_1018(Node_2511531018 node) {
    int count_1018 = 1;
    if (node == null) {
        return 0;
    } else {
        count_1018 +=
countNodes_1018(node.getLeft_1018());
        count_1018 +=
countNodes_1018(node.getRight_1018());
        return count_1018;
    }
}

```

Gambar 2.2.2.8 Menghitung node.

- 2.2.2.9 Digunakan untuk menampilkan struktur tree dalam bentuk visual menggunakan method `print` dan mengambil node yang sedang aktif atau sedang diproses.

```

public void print_1018() {
    root_1018.print_1018();
}

```

```
public Node_2511531018 getCurrent_1018() {
    return currentNode_1018;
}
```

Gambar 2.2.2.9 Method print dan Method getCurrent.

2.2.2.10 Mengubah nilai currentNode menjadi node yang diberikan.

Method setRoot digunakan untuk menentukan atau mengganti root dari tree.

```
public void setCurrent_1018(Node_2511531018 node) {
    this.currentNode_1018 = node;
}
public void setRoot_1018(Node_2511531018 root_1018)
{
    this.root_1018 = root_1018;
}
```

Gambar 2.2.2.10 Method setCurrent dan Method setRoot.

2.3 BtreeDriver_2511531018

2.3.1 Kode Program

```
package pekan9_2511531018;

public class BtreeDriver_2511531018 {
    public static void main(String[] args) {
        //Membuat Pohon
        BTree_2511531018 tree_1018 = new BTree_2511531018();
        System.out.print("Jumlah Simpul awal pohon: ");
        System.out.println(tree_1018.countNodes_1018());
        //menambahkan simpul data 1
        Node_2511531018 root_1018 = new Node_2511531018(1);
        //menjadikan simpul 1 sebagai root
        tree_1018.setRoot_1018(root_1018);
        System.out.println("Jumlah simpul jika hanya ada
root");
        System.out.println(tree_1018.countNodes_1018());
        Node_2511531018 node2_1018 = new Node_2511531018(2);
        Node_2511531018 node3_1018 = new Node_2511531018(3);
        Node_2511531018 node4_1018 = new Node_2511531018(4);
        Node_2511531018 node5_1018 = new Node_2511531018(5);
        Node_2511531018 node6_1018 = new Node_2511531018(6);
        Node_2511531018 node7_1018 = new Node_2511531018(7);
        Node_2511531018 node8_1018 = new Node_2511531018(8);
        Node_2511531018 node9_1018 = new Node_2511531018(9);
        root_1018.setLeft_1018(node2_1018);
        node2_1018.setLeft_1018(node4_1018);
        node2_1018.setRight_1018(node5_1018);
        node4_1018.setRight_1018(node8_1018);
        root_1018.setRight_1018(node3_1018);
        node3_1018.setLeft_1018(node6_1018);
        node3_1018.setRight_1018(node7_1018);
    }
}
```

```

        node6_1018.setLeft_1018(node9_1018);

        //Set root
        tree_1018.setCurrent_1018(tree_1018.getRoot_1018());
        System.out.println("menampilkan simpul terakhir");

        System.out.println(tree_1018.getCurrent_1018().getData_1018());
        System.out.println("Jumlah sumpul; setelah simpul 7
        ditambahkan");
        System.out.println(tree_1018.countNodes_1018());
        System.out.println("Inorder: ");
        tree_1018.printInorder_1018();
        System.out.println("\nPreorder: ");
        tree_1018.printPreorder_1018();
        System.out.println("\nPostorder: ");
        tree_1018.printPostOrder_1018();
        System.out.println("\nmenampilkan simpul dalam
        bentuk pohon");
        tree_1018.print_1018();
    }
}

```

Gambar 2.3.1 Kode program BtreeDriver_2511531018.

2.3.2 Langkah Kerja

- 2.3.2.1 Membuat binary tree baru yang masih kosong karena belum memiliki root. Kemudian program menampilkan jumlah simpul awal. Karena tree masih kosong, method countNodes akan menghasilkan 0.

```

public class BtreeDriver_2511531018 {
    public static void main(String[] args) {
        //Membuat Pohon
        BTree_2511531018 tree_1018 = new
        BTree_2511531018();
        System.out.print("Jumlah Simpul awal pohon:
        ");

        System.out.println(tree_1018.countNodes_1018());
    }
}

```

Gambar 2.3.2.1 Membuat binary tree.

- 2.3.2.2 Selanjutnya dibuat node pertama yang berisi nilai 1, node ini nantinya akan menjadi akar (root) dari tree. Baris tersebut menjadikan node bernilai 1 sebagai root dari Binary Tree. Setelah root ditambahkan, jumlah simpul dihitung kembali. Karena hanya ada satu node yaitu root, hasilnya menjadi 1.

```

Node_2511531018 root_1018 = new Node_2511531018(1);

```

```
//menjadikan simpul 1 sebagai root
tree_1018.setRoot_1018(root_1018);
System.out.println("Jumlah simpul jika hanya
ada root");
System.out.println(tree_1018.countNodes_1018());
```

Gambar 2.3.2.2 Membuat node.

- 2.3.2.3 Membuat node-node lain yang berisi data 2 sampai 9. Masing-masing node dibuat sebagai objek terpisah agar nantinya dapat dihubungkan menjadi sebuah pohon.

```
Node_2511531018 node2_1018 = new Node_2511531018(2);
Node_2511531018 node3_1018 = new
Node_2511531018(3);
Node_2511531018 node4_1018 = new
Node_2511531018(4);
Node_2511531018 node5_1018 = new
Node_2511531018(5);
Node_2511531018 node6_1018 = new
Node_2511531018(6);
Node_2511531018 node7_1018= new
Node_2511531018(7);
Node_2511531018 node8_1018 = new
Node_2511531018(8);
Node_2511531018 node9_1018 = new
Node_2511531018(9);
```

Gambar 2.3.2.3 Membuat node-node lain.

- 2.3.2.4 Setelah semua node dibuat, program mulai menyusun struktur Binary Tree. Node 2 menjadi anak kiri root dan node 3 menjadi anak kanan root. Node 4 dan 5 menjadi anak dari node 2. Node 8 menjadi anak kanan node 4. Node 6 dan 7 menjadi anak dari node 3. Node 9 menjadi anak kiri node 6.

```
root_1018.setLeft_1018(node2_1018);
node2_1018.setLeft_1018(node4_1018);
node2_1018.setRight_1018(node5_1018);
node4_1018.setRight_1018(node8_1018);
root_1018.setRight_1018(node3_1018);
node3_1018.setLeft_1018(node6_1018);
node3_1018.setRight_1018(node7_1018);
node6_1018.setLeft_1018(node9_1018);
```

Gambar 2.3.2.4 Menyusun struktur Binary Tree.

- 2.3.2.5 Kemudian root disimpan sebagai current node (node yang sedang aktif adalah node root), karena current node adalah root, maka yang ditampilkan adalah nilai 1. Selanjutnya

menghitung jumlah seluruh simpul. Karena terdapat node 1 sampai 9, jumlah simpul yang dihitung adalah 9.

```
tree_1018.setCurrent_1018(tree_1018.getRoot_1018());
    System.out.println("menampilkan simpul
    terakhir");

    System.out.println(tree_1018.getCurrent_1018().getDa
    ta_1018());
    System.out.println("Jumlah sumpul; setelah
    simpul 7 ditambahkan");

    System.out.println(tree_1018.countNodes_1018());
    System.out.println("Inorder: ");
```

Gambar 2.3.2.5 Current Node.

2.3.2.6 Kemudian melakukan traversal Inorder, Preorder, dan Postorder. Setelah itu, menampilkan simpul dalam bentuk pohon.

```
tree_1018.printInorder_1018();
    System.out.println("\nPreorder: ");
    tree_1018.printPreorder_1018();
    System.out.println("\nPostorder: ");
    tree_1018.printPostOrder_1018();
    System.out.println("\nmenampilkan simpul
    dalam bentuk pohon");
    tree_1018.print_1018();
```

gambar 2.3.2.6 Traversal Tree.

2.4.3 Output

```
Jumlah Simpul awal pohon: 0
Jumlah simpul jika hanya ada root
1
menampilkan simpul terakhir
1
Jumlah sumpul; setelah simpul 7 ditambahkan
9
Inorder:
4 8 2 5 1 9 6 3 7
Preorder:
1 2 4 8 5 3 6 9 7
Postorder:
8 4 5 2 9 6 7 3 1
menampilkan simpul dalam bentuk pohon
|
|-- 7
|   |-- 3
|   |   |-- 6
|   |   |-- 9
|-- 1
    |-- 5
    |-- 2
        |-- 8
        |-- 4
```

Gambar 2.3.3 Output BtreeDriver_2511531018.

2.4 GraphTraversal_2511531018

2.4.1 Kode Program

```
package pekan9_2511531018;

import java.util.*;

public class GraphTraversal_2511531018 {
    private Map<String, List<String>> graph_1018 = new
    HashMap<>();

    // Menambahkan edge (graf tak berarah)
    public void addEdge(String node1, String node2) {
        graph_1018.putIfAbsent(node1, new ArrayList<>());
        graph_1018.putIfAbsent(node2, new ArrayList<>());
        graph_1018.get(node1).add(node2);
        graph_1018.get(node2).add(node1);
    }

    // Menampilkan graf awal
    public void printGraph() {
        System.out.println("Graf Awal (Adjacency List):");
        for (String node_1018 : graph_1018.keySet()) {
            System.out.print(node_1018 + " -> ");
            List<String> neighbors =
graph_1018.get(node_1018);
            System.out.println(String.join(", ",
neighbors));
        }
        System.out.println();
    }

    // DFS rekursif
    public void dfs_1018(String start_1018) {
        Set<String> visited_1018 = new HashSet<>();
        System.out.println("Penelusuran DFS:");
        dfsHelper_1018(start_1018, visited_1018);
        System.out.println();
    }

    private void dfsHelper_1018(String current_1018,
Set<String> visited) {
        if (visited.contains(current_1018)) return;
        visited.add(current_1018);
        System.out.print(current_1018 + " ");
        for (String neighbor :
graph_1018.getDefault(current_1018, new ArrayList<>())) {
            dfsHelper_1018(neighbor, visited);
        }
    }

    // BFS iteratif
    public void bfs_1018(String start_1018) {
        Set<String> visited_1018 = new HashSet<>();
        Queue<String> queue_1018 = new LinkedList<>();
        queue_1018.add(start_1018);
        visited_1018.add(start_1018);
    }
}
```

```

        System.out.println("Penelusuran BFS: ");
        while (!queue_1018.isEmpty()) {
            String current_1018 = queue_1018.poll();
            System.out.print(current_1018 + " ");
            for (String neighbor :
graph_1018.getDefault(current_1018, new ArrayList<>())) {
                if (!visited_1018.contains(neighbor)) {
                    queue_1018.add(neighbor);
                    visited_1018.add(neighbor);
                }
            }
        }
        System.out.println();
    }
    // Main
    public static void main(String[] args) {
        GraphTraversal_2511531018 graph_1018 = new
GraphTraversal_2511531018();

        //Contoh graf: A-B, A-C, B-D, B-E
        graph_1018.addEdge("A", "B");
        graph_1018.addEdge("A", "C");
        graph_1018.addEdge("B", "D");
        graph_1018.addEdge("B", "E");
        //Cetak graf awal
        System.out.println("Graf Awal adalah: ");
        graph_1018.printGraph();
        //Lakukan penelusuran
        graph_1018.dfs_1018("A");
        graph_1018.bfs_1018("A");
    }
}

```

Gambar 2.4.1 Kode program GraphTraversal_2511531018.

2.4.2 Langkah Kerja

2.4.2.1 Graf disimpan menggunakan Adjacency List, string menyimpan nama simpul (A, B, C, dan seterusnya), List<String> menyimpan daftar tetangga dari setiap simpul, dan HashMap digunakan untuk agar pencarian data lebih cepat.

```

public class GraphTraversal_2511531018 {
    private Map<String, List<String>> graph_1018 = new
HashMap<>();

```

Gambar 2.4.2.1 Deklarasi Graf.

2.4.2.2 Digunakan untuk menambahkan hubungan antar simpul, jika simpul belum ada di graf, maka akan dibuat terlebih dahulu.

Karena graf bersifat tak berarah, maka hubungan dibuat dua arah.

```
public void addEdge(String node1, String node2) {
    graph_1018.putIfAbsent(node1, new
    ArrayList<>());
    graph_1018.putIfAbsent(node2, new
    ArrayList<>());
    graph_1018.get(node1).add(node2);
    graph_1018.get(node2).add(node1);
}
```

Gambar 2.4.2.2 Method addEdge.

- 2.4.2.3 Digunakan untuk menampilkan isi graf dalam bentuk adjacency list. Perulangan digunakan untuk mengakses seluruh simpul yang ada di graf. Menggabungkan daftar tetangga menjadi satu string yang dipisahkan koma,

```
public void printGraph() {
    System.out.println("Graf Awal (Adjacency
    List):");
    for (String node_1018 : graph_1018.keySet())
    {
        System.out.print(node_1018 + " -> ");
        List<String> neighbors =
        graph_1018.get(node_1018);
        System.out.println(String.join(", ",
        neighbors));
    }
    System.out.println();
}
```

Gambar 2.4.2.3 Method printGraph.

- 2.4.2.4 Digunakan untuk memulai penelusuran DFS dari simpul awal. Menyimpan simpul yang sudah dikunjungi agar tidak terjadi pengulangan dan memanggil method rekursif yang melakukan proses DFS.

```
public void dfs_1018(String start_1018) {
    Set<String> visited_1018 = new HashSet<>();
    System.out.println("Penelusuran DFS:");
    dfsHelper_1018(start_1018, visited_1018);
    System.out.println();
}
```

Gambar 2.4.2.4 Method DFS.

- 2.4.2.5 Jika simpul sudah pernah dikunjungi, maka proses dihentikan, menandai simpul yang sudah dikunjungi, menampilkan simpul yang sedang dikunjungi, mengunjungi seluruh tetangga dari simpul saat ini. DFS bergerak terus ke daslam hingga

mencapai simpul paling akhir sebelum kembali ke simpul sebelumnya.

```
private void dfsHelper_1018(String current_1018,
Set<String> visited) {
    if (visited.contains(current_1018)) return;
    visited.add(current_1018);
    System.out.print(current_1018 + " ");
    for (String neighbor :
graph_1018.getOrDefault(current_1018, new
ArrayList<>())) {
        dfsHelper_1018(neighbor, visited);
    }
}
```

Gambar 2.4.2.5 Method DFS Helper.

- 2.4.2.6 Method ini digunakan untuk melakukan penelusuran BFS. Visited menyimpan simpul yang sudah dikunjungi, queue digunakan untuk menyimpan simpul yang akan diproses. Simpul awal dimasukkan ke dalam queue dan ditandai sudah dikunjungi. Perulangan berjalan selama queue masih memiliki isi.

```
public void bfs_1018(String start_1018) {
    Set<String> visited_1018 = new HashSet<>();
    Queue<String> queue_1018 = new
LinkedList<>();
    queue_1018.add(start_1018);
    visited_1018.add(start_1018);
    System.out.println("Penelusuran BFS: ");
    while (!queue_1018.isEmpty()) {
```

Gambar 2.4.2.6 Method BFS..

- 2.4.2.7 Mengambil simpul paling depan dari queue, menampilkan simpul, mengakses semua tetangga dari simpul saat ini. Jika tetangga belum pernah dikunjungi, tetangga dimasukkan ke queue dan ditandai sudah dikunjungi.

```
String current_1018 = queue_1018.poll();
    System.out.print(current_1018 + " ");
    for (String neighbor :
graph_1018.getOrDefault(current_1018, new
ArrayList<>())) {
        if
(!visited_1018.contains(neighbor)) {
            queue_1018.add(neighbor);
        }
        visited_1018.add(neighbor);
    }
}
```

Gambar 2.4.2.7 Perulangan BFS.

- 2.4.2.8 Membuat objek graf.

```
public static void main(String[] args) {
    GraphTraversal_2511531018 graph_1018 = new
    GraphTraversal_2511531018();
}
```

Gambar 2.4.2.8 Method main.

2.4.2.9 Menambahkan edge dan menampilkan graf.

```
graph_1018.addEdge("A", "B");
graph_1018.addEdge("A", "C");
graph_1018.addEdge("B", "D");
graph_1018.addEdge("B", "E");
//Cetak graf awal
System.out.println("Graf Awal adalah: ");
graph_1018.printGraph();
//Lakukan penelusuran
graph_1018.dfs_1018("A");
graph_1018.bfs_1018("A");
```

Gambar 2.4.2.9 Menampilkan graf.

2.4.3 Output

```
Graf Awal adalah:
Graf Awal (Adjacency List):
A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B
|
Penelusuran DFS:
A B D E C
Penelusuran BFS:
A B C D E
```

Gambar 2.4.3 Output GraphTraversal_2511531018.

2.5 TUGAS

2.6.1 PetaMall_2511531018

```
package pekan9_2511531018;

import javax.swing.*;
import java.awt.*;
import java.util.*;

public class PetaMall_2511531018 extends JFrame {

    private JComboBox<String> cmbAwal_1018;
    private JComboBox<String> cmbTujuan_1018;

    private JButton btnBFS_1018;
    private JButton btnDFS_1018;
    private JButton btnReset_1018;

    private JLabel lblJalur_1018;
    private JLabel lblNode_1018;
    private JLabel lblJumlah_1018;

    private GraphPanel_1018 panelGraph_1018;

    private HashMap<String, ArrayList<String>> graph_1018;

    public PetaMall_2511531018() {

        setTitle("PENCARIAN JALUR MALL MENGGUNAKAN BFS DAN DFS");
        setSize(1000,700);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(EXIT_ON_CLOSE);

        initGraph();

        JPanel top = new JPanel();

        cmbAwal_1018 = new
JComboBox<>(graph_1018.keySet().toArray(new String[0]));
        cmbTujuan_1018 = new
JComboBox<>(graph_1018.keySet().toArray(new String[0]));

        btnBFS_1018 = new JButton("BFS");
        btnDFS_1018 = new JButton("DFS");
        btnReset_1018 = new JButton("RESET");

        top.add(new JLabel("Lokasi Awal"));
        top.add(cmbAwal_1018);

        top.add(new JLabel("Lokasi Tujuan"));
        top.add(cmbTujuan_1018);

        top.add(btnBFS_1018);
        top.add(btnDFS_1018);
        top.add(btnReset_1018);
    }
}
```

```

        add(top, BorderLayout.NORTH);

        panelGraph_1018 = new GraphPanel_1018();
        add(panelGraph_1018, BorderLayout.CENTER);

        JPanel bottom = new JPanel();
        bottom.setLayout(new GridLayout(3,1));

        lblJalur_1018 = new JLabel("Jalur : ");
        lblNode_1018 = new JLabel("Node Dikunjungi : ");
        lblJumlah_1018 = new JLabel("Jumlah Node Dikunjungi : 
0");

        bottom.add(lblJalur_1018);
        bottom.add(lblNode_1018);
        bottom.add(lblJumlah_1018);

        add(bottom, BorderLayout.SOUTH);

        btnBFS_1018.addActionListener(e -> BFS_1018());
        btnDFS_1018.addActionListener(e -> DFS_1018());
        btnReset_1018.addActionListener(e ->
resetGraph_1018());

        setVisible(true);
    }

    private void initGraph() {

        graph_1018 = new HashMap<>();

        addEdge("Pintu Utama","Customer Service");
        addEdge("Pintu Utama","ATM Center");

        addEdge("Customer Service","Food Court");
        addEdge("Customer Service","Lift");

        addEdge("Food Court","Bioskop");
        addEdge("Food Court","Mushola");

        addEdge("Bioskop","Area Bermain");
        addEdge("Bioskop","Lift");

        addEdge("Lift","Toko Fashion");
        addEdge("Lift","Toko Elektronik");

        addEdge("Toko Fashion","Supermarket");
        addEdge("Area Bermain","Toko Fashion");

        addEdge("Toko Elektronik","Supermarket");

        addEdge("ATM Center","Mushola");
        addEdge("ATM Center","Parkiran");

        addEdge("Parkiran","Pintu Utama");
    }

```

```

private void addEdge(String a,String b){

    graph_1018.putIfAbsent(a,new ArrayList<>());
    graph_1018.putIfAbsent(b,new ArrayList<>());

    graph_1018.get(a).add(b);
    graph_1018.get(b).add(a);
}

private void BFS_1018(){

    String start =
cmbAwal_1018.getSelectedItemAt().toString();
    String goal =
cmbTujuan_1018.getSelectedItemAt().toString();

    Queue<String> queue = new LinkedList<>();
    Set<String> visited = new HashSet<>();
    HashMap<String,String> parent = new HashMap<>();

    ArrayList<String> urutan = new ArrayList<>();

    queue.add(start);
    visited.add(start);

    while(!queue.isEmpty()){

        String current = queue.poll();

        urutan.add(current);

        if(current.equals(goal))
            break;

        for(String next : graph_1018.get(current)){

            if(!visited.contains(next)){

                visited.add(next);
                parent.put(next,current);
                queue.add(next);
            }
        }
    }

    tampilkanHasil(goal,parent,urutan);
}

private void DFS_1018(){

    String start =
cmbAwal_1018.getSelectedItemAt().toString();
    String goal =
cmbTujuan_1018.getSelectedItemAt().toString();

    Stack<String> stack = new Stack<>();

```

```

Set<String> visited = new HashSet<>();
HashMap<String,String> parent = new HashMap<>();

ArrayList<String> urutan = new ArrayList<>();

stack.push(start);

while(!stack.isEmpty()){

    String current = stack.pop();

    if(visited.contains(current))
        continue;

    visited.add(current);

    urutan.add(current);

    if(current.equals(goal))
        break;

    ArrayList<String> tetangga =
graph_1018.get(current);

    for(int i=tetangga.size()-1;i>=0;i--){

        String next = tetangga.get(i);

        if(!visited.contains(next)){

            parent.put(next,current);
            stack.push(next);

        }

    }

    tampilkanHasil(goal,parent,urutan);
}

private void tampilkanHasil(String goal,
                             HashMap<String,String>
parent,
                             ArrayList<String> urutan){

    ArrayList<String> path = new ArrayList<>();

    String current = goal;

    while(current != null){

        path.add(current);
        current = parent.get(current);
    }

    Collections.reverse(path);

    lblJalur_1018.setText("Jalur : " + path);

```

```

        lblNode_1018.setText("Node Dikunjungi : " + urutan);
        lblJumlah_1018.setText("Jumlah Node Dikunjungi : " +
urutan.size());

        panelGraph_1018.setVisited(urutan,path);
    }

    private void resetGraph_1018(){

        lblJalur_1018.setText("Jalur : ");
        lblNode_1018.setText("Node Dikunjungi : ");
        lblJumlah_1018.setText("Jumlah Node Dikunjungi : 0");

        panelGraph_1018.reset();
    }

    class GraphPanel_1018 extends JPanel{

        HashMap<String,Point> posisi = new HashMap<>();

        ArrayList<String> visited = new ArrayList<>();
        ArrayList<String> path = new ArrayList<>();

        public GraphPanel_1018(){

            posisi.put("Bioskop",new Point(400,70));
            posisi.put("Area Bermain",new Point(650,70));

            posisi.put("Food Court",new Point(220,180));
            posisi.put("Toko Fashion",new Point(650,180));

            posisi.put("Customer Service",new
Point(220,300));
            posisi.put("Lift",new Point(450,300));
            posisi.put("Toko Elektronik",new Point(700,300));

            posisi.put("Pintu Utama",new Point(220,450));
            posisi.put("ATM Center",new Point(450,450));
            posisi.put("Supermarket",new Point(700,450));

            posisi.put("Parkiran",new Point(220,560));
            posisi.put("Mushola",new Point(450,560));
        }

        public void setVisited(ArrayList<String>
v,ArrayList<String> p){

            visited = v;
            path = p;
            repaint();
        }

        public void reset(){

            visited.clear();
            path.clear();
            repaint();
        }
    }

```

```

    }

    protected void paintComponent(Graphics g){
        super.paintComponent(g);

        Graphics2D g2 = (Graphics2D) g;

        for(String a : graph_1018.keySet()){
            for(String b : graph_1018.get(a)){
                Point p1 = posisi.get(a);
                Point p2 = posisi.get(b);

                g2.drawLine(
                    p1.x,p1.y,
                    p2.x,p2.y);
            }
        }

        for(String node : posisi.keySet()){
            Point p = posisi.get(node);

            if(path.contains(node))
                g2.setColor(Color.GREEN);

            else if(visited.contains(node))
                g2.setColor(Color.ORANGE);

            else
                g2.setColor(Color.LIGHT_GRAY);

            g2.fillOval(p.x-25,p.y-25,50,50);

            g2.setColor(Color.BLACK);
            g2.drawOval(p.x-25,p.y-25,50,50);

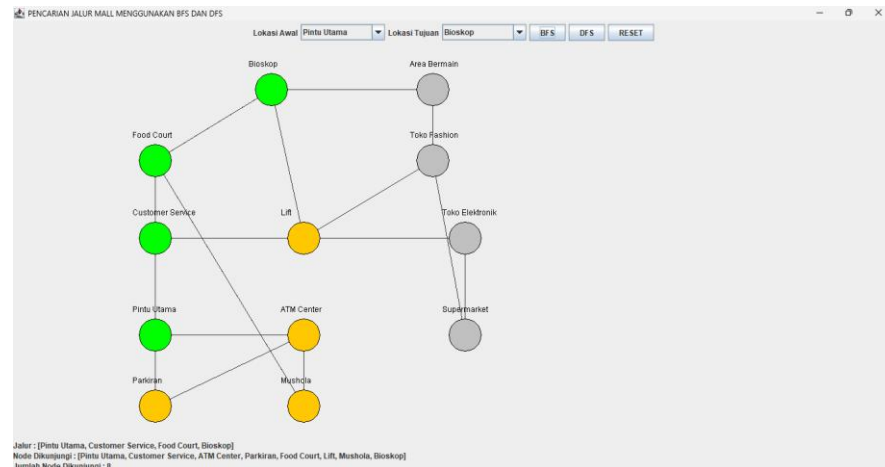
            g2.drawString(node,p.x-35,p.y-35);
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(
            () -> new PetaMall_2511531018()
        );
    }
}

```

Gambar 2.6.1 Kode program PetaMall_2511531018.

2.6.2 Output



Gambar 2.6.3 Output kode program PetaMall_2511531018.

2.6.3 Penjelasan

1. Deskripsi Peta

Peta Mall dibuat dalam bentuk graf tidak berarah, dimana tiap lokasi disebut dengan node dan tiap sisi yang menghubungkan antar node disebut dengan edge.

2. Jumlah Node dan Edge

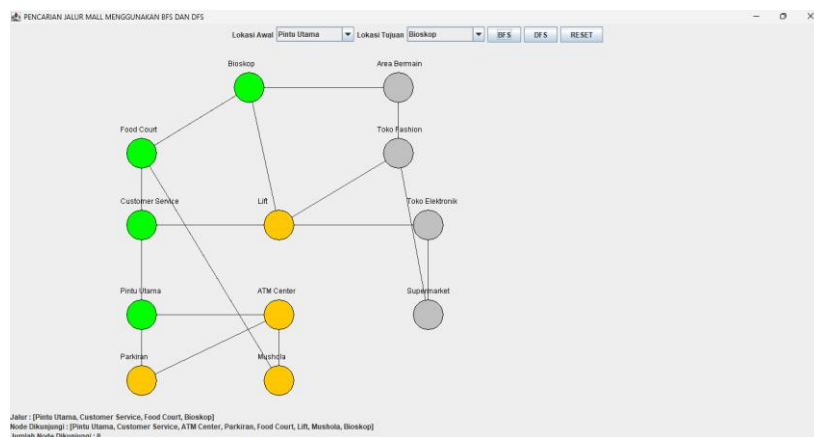
Jumlah node: 12

- Pintu Utama
- Customer Service
- Food Court
- Bioskop
- Area Bermain
- Toko Fashion
- Toko Elektronik
- Supermarket
- Lift
- ATM Center
- Mushola
- Parkiran

Jumlah edge: 16

- Pintu Utama ↔ Customer Service
- Pintu Utama ↔ ATM Center
- Pintu Utama ↔ Parkiran
- Customer Service ↔ Food Court
- Customer Service ↔ Lift
- Food Court ↔ Bioskop
- Food Court ↔ Mushola
- Bioskop ↔ Area Bermain
- Bioskop ↔ Lift
- Area Bermain ↔ Toko Fashion
- Toko Fashion ↔ Toko Elektronik
- Toko Fashion ↔ Lift
- Toko Elektronik ↔ Lift
- Toko Elektronik ↔ Supermarket
- ATM Center ↔ Mushola
- ATM Center ↔ Parkiran

3. Hasil BFS

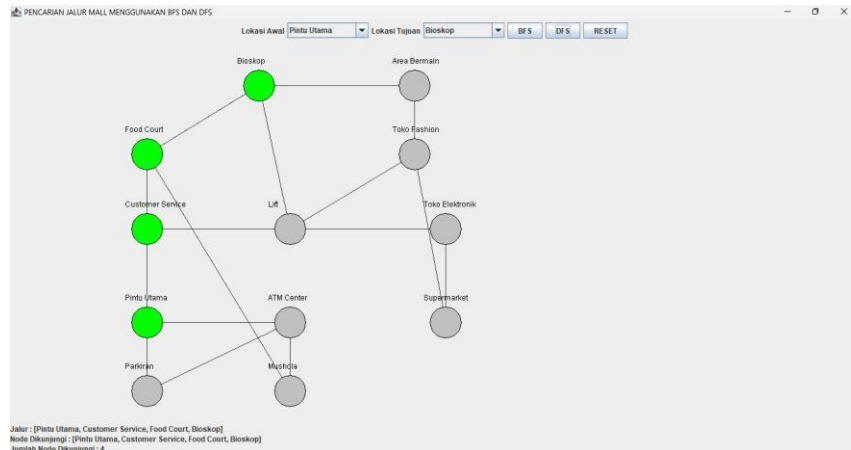


Jalur: Pintu Utama → Customer Service → Food Court → Bioskop.

Node yang dikunjungi: Pintu Utama, Parkiran, ATM Center, Musholla, Food Court, Customer Service, Lift, dan Bioskop.

Jumlah node yang dikunjungi: 8

4. Hasil DFS



Jalur: Pintu Utama → Customer Service → Food Court → Bioskop.

Node yang dikunjungi: Pintu Utama, Customer Service, Food Court, dan Bioskop.

Jumlah node yang dikunjungi: 4

5. Kesimpulan

Pada pencarian dari Pintu Utama menuju Bioskop, BFS mengunjungi lebih banyak node karena algoritma ini menelusuri seluruh node pada level yang sama terlebih dahulu. Sebaliknya, DFS langsung menelusuri satu cabang hingga menemukan tujuan sehingga jumlah node yang dikunjungi dapat lebih sedikit. Namun BFS menjamin jalur yang ditemukan merupakan jalur terpendek, sedangkan DFS belum tentu menghasilkan jalur terpendek

BAB III PENUTUP

KESIMPULAN

Binary Tree, setiap node dapat dihubungkan menjadi sebuah pohon dengan root sebagai simpul utama, kemudian dilakukan traversal menggunakan metode Inorder, Preorder, dan Postorder untuk mengunjungi seluruh simpul pada tree. Selain itu, dilakukan pula proses pencarian data dan perhitungan jumlah simpul yang terdapat pada pohon. Pada Graph, hubungan antar simpul direpresentasikan menggunakan adjacency list sehingga memudahkan proses penyimpanan dan penelusuran data. Penelusuran graf dilakukan menggunakan algoritma DFS (Depth First Search) dan BFS (Breadth First Search). DFS menelusuri simpul hingga kedalaman tertentu terlebih dahulu sebelum kembali ke simpul sebelumnya, sedangkan BFS menelusuri simpul berdasarkan level atau tingkat kedekatan dari simpul awal.

DAFTAR PUSTAKA

- [1] Rosa A.S dan M. Shalahuddin, *Rekayasa Perangkat Lunak Terstruktur dan Berorientasi Objek*, Bandung: Informatika, 2018.